CRYSTALS–Kyber:
a CCA-secure module-lattice-based KEM

송백경
baekkyung777@korea.ac.kr

Contents

- Contribution
- Preliminaries
- Kyber's IND-CPA-secure encryption
- The CCA-secure KEM
- Key Exchange Protocols

Contribution

Contribution

- Our main contribution is a highly-optimized instantiation of a **CCA-secure KEM** called Kyber, which is based on the **hardness of Module-LWE**.
  - **efficient** as the ones based on Ring-LWE
  - **flexibility** and **security advantages**
    - $R_q = \mathbb{Z}_{7681}[X]/(X^{256}+1)$ k=2 : 102-bit / k=3 128-bit

- More precisely, we instantiate a CPA-secure PKE scheme **Kyber.CPA** in Section 3,

- then apply a variant of the **Fujisaki–Okamoto transform** to create a **CCA-Secure KEM** Kyber in Section 4.

Preliminaries

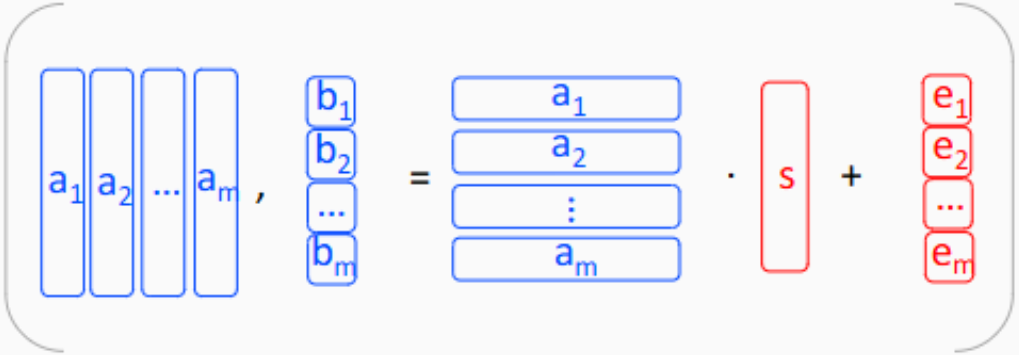
MLWE

- **LWE**

Given some uniform random a, $b = a \cdot s + e$:

- **(search LWE)** decode the noisy product b i.e. recover s from b for "small" e
- **(decision LWE)** distinguish b from uniform random

Plain LWE sample: $a \leftarrow \mathbb{Z}_q^n$; $s \leftarrow U$ or χ_σ^n, $e \leftarrow \chi_\sigma$; $b \in \mathbb{Z}_q$



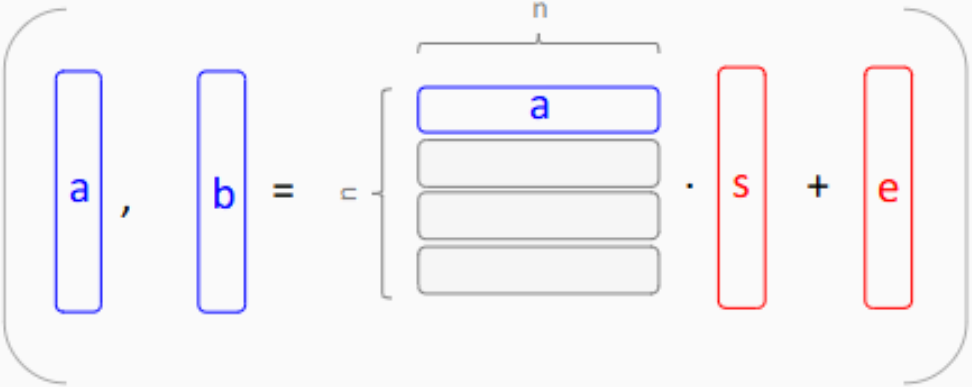
MLWE

- **Ring-LWE**

Let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. Given some uniform random $a \in R_q$,

- **(search)** recover $s \in R_q$ from $b = a \cdot s + e$ for "small" $e \in R_q$
- **(decision)** decide whether $b = a \cdot s + e$ or b is random

Error distribution: $s, e \leftarrow \chi_\sigma^n$



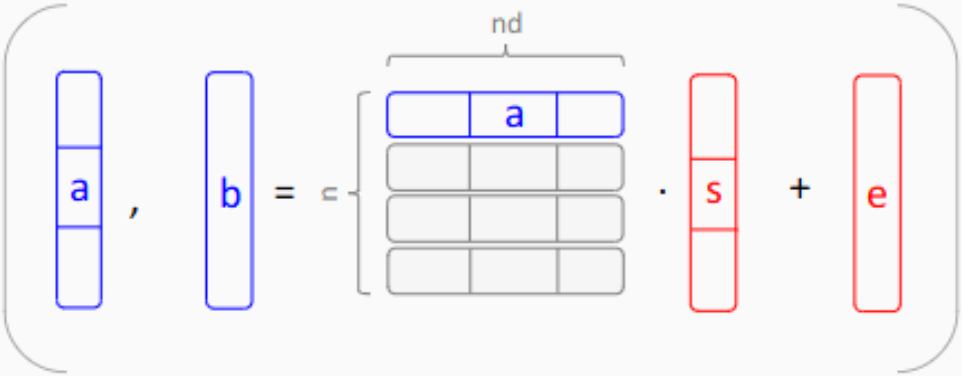
MLWE

- **Module-LWE**

Given some uniform random $a \in (R_q)^d$,

- **(search)** recover $s \in (R_q)^d$ from $b = \mathbf{a} \cdot \mathbf{s} + e$ for "small" $e \in R_q$
- **(decision)** decide whether $b = \mathbf{a} \cdot \mathbf{s} + e$ or b is random

Error distribution: $\mathbf{s} \leftarrow \chi_\sigma^{nd}, e \leftarrow \chi_\sigma^n$



Compression and Decompression

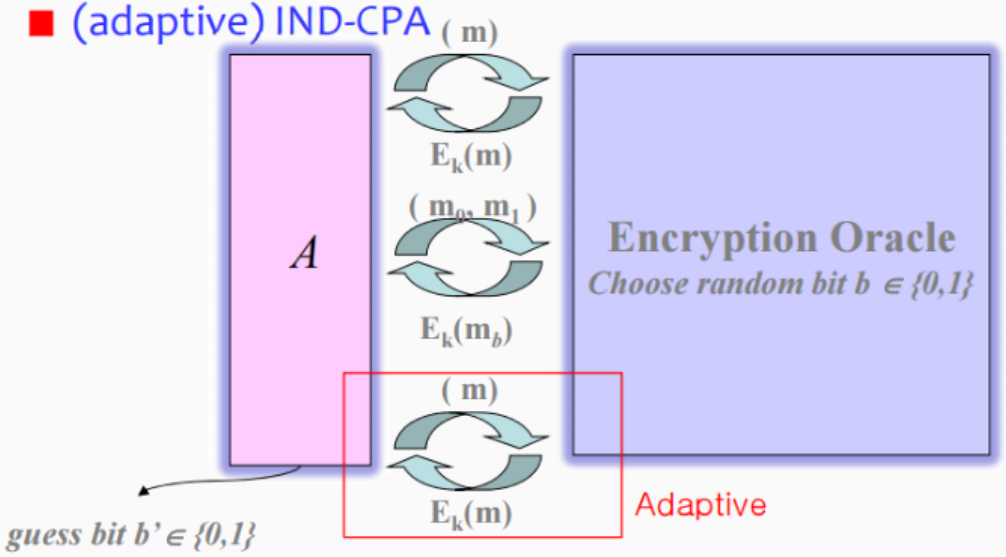
- **Compression and Decompression**

$\mathsf{Compress}_q(x, d)$ that takes an element $x \in \mathbb{Z}_q$ and outputs an integer in $\{0, \ldots, 2^d - 1\}$, where $d < \lceil \log_2(q) \rceil$.

$$\mathsf{Compress}_q(x, d) = \lceil (2^d/q) \cdot x \rfloor \bmod^+ 2^d,$$
$$\mathsf{Decompress}_q(x, d) = \lceil (q/2^d) \cdot x \rfloor.$$

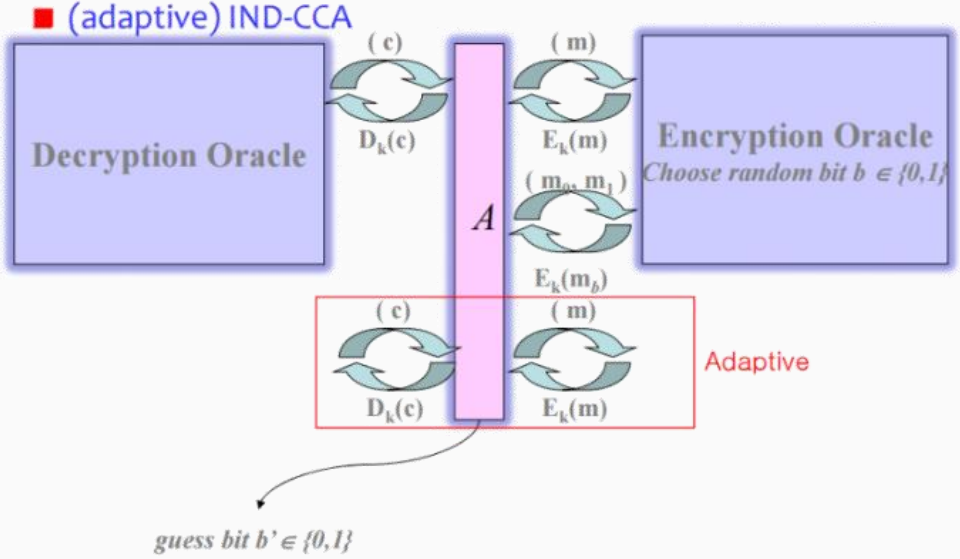
IND-CPA and IND-CCA

- **IND-CPA(선택된 평문 공격에 대한 구별 불가능성)**
  - 시나리오 : IND-CPA에서 공격자는 임의의 평문을 선택하고 해당 평문에 해당하는 암호문을 얻을 수 있습니다. 그러나 공격자는 복호화 오라클에 접근할 수 없습니다. 즉, 어떤 암호문도 복호화할 수 없습니다.



IND-CPA and IND-CCA

- **IND-CCA(선택된 암호문 공격에 대한 구별 불가능성)**
  - 시나리오 : IND-CCA에서 공격자는 암호화할 평문을 선택할 수 있을 뿐만 아니라, 임의의 암호문(챌린지 암호문 제외)을 복호화할 수도 있습니다. 즉, 공격자는 복호화 오라클을 사용하여 이전에 암호화된 메시지에서 정보를 수집할 수 있습니다.



Cryptographic definitions

- **PKE correct**
  - public-key encryption scheme PKE is $(1-\delta)$-correct if

$$\mathbf{E}[\max_{m\in\mathcal{M}} \Pr[\mathsf{Dec}(sk, \mathsf{Enc}(pk, m)) = m]] \geq 1-\delta,$$

- **Advantage of an adversary**

$$\mathbf{Adv}^{\mathrm{cca}}_{\mathsf{PKE}}(\mathsf{A}) = \left| \Pr\left[b = b' \;:\; \begin{array}{l} (pk, sk) \leftarrow \mathsf{KeyGen}(); \\ (m_0, m_1, s) \leftarrow \mathsf{A}^{\mathrm{DEC}(\cdot)}(pk); \\ b \leftarrow \{0,1\}; c^* \leftarrow \mathsf{Enc}(pk, m_b); \\ b' \leftarrow \mathsf{A}^{\mathrm{DEC}(\cdot)}(s, c^*) \end{array} \right] - \frac{1}{2} \right|,$$

where the decryption oracle is defined as $\mathrm{DEC}(\cdot) := \mathsf{Dec}(sk, \cdot)$. We further require that $|m_0| = |m_1|$ and that in the second phase A is not allowed to query $\mathrm{DEC}(\cdot)$ with the challenge ciphertext c^*. The advantage $\mathbf{Adv}^{\mathrm{cpa}}_{\mathsf{PKE}}(\mathsf{A})$ of an adversary A is defined as $\mathbf{Adv}^{\mathrm{cca}}_{\mathsf{PKE}}(\mathsf{A})$, with the modification that A cannot query the decryption oracle.

Cryptographic definitions

- **Module-LWE advantage**

$$\mathbf{Adv}^{\mathrm{mlwe}}_{m,k,\eta}(\mathsf{A}) =$$

$$\left|\Pr\left[b' = 1 \ : \ \begin{array}{l}\mathbf{A} \leftarrow R_q^{m\times k}; (\mathbf{s},\mathbf{e}) \leftarrow \beta_\eta^k \times \beta_\eta^m; \\ \mathbf{b} = \mathbf{As} + \mathbf{e}; b' \leftarrow \mathsf{A}(\mathbf{A},\mathbf{b})\end{array}\right]\right.$$
$$\left. - \Pr\left[b' = 1 \ : \ \mathbf{A} \leftarrow R_q^{m\times k}; \mathbf{b} \leftarrow R_q^m; b' \leftarrow \mathsf{A}(\mathbf{A},\mathbf{b})\right]\right|.$$

Kyber's IND-CPA-secure encryption

Kyber.CPA

Let k, d_t, d_u, d_v be positive integer parameters, and recall that $n = 256$. Let $\mathcal{M} = \{0,1\}^{256}$ denote the message space, where every message $m \in \mathcal{M}$ can be viewed as a polynomial in R with coefficients in $\{0,1\}$. Consider the public-key encryption scheme Kyber.CPA $=$ (KeyGen, Enc, Dec) as described in Algorithms 1 to 3. Note that cipheretxts are of the form $(\mathbf{u}, v) \in \{0,1\}^{256 \cdot k d_u} \times \{0,1\}^{256 \cdot d_v}$.

Algorithm 1 Kyber.CPA.KeyGen(): key generation

1: $\rho, \sigma \leftarrow \{0,1\}^{256}$
2: $\mathbf{A} \sim R_q^{k \times k} := \mathsf{Sam}(\rho)$
3: $(\mathbf{s}, \mathbf{e}) \sim \beta_\eta^k \times \beta_\eta^k := \mathsf{Sam}(\sigma)$
4: $\mathbf{t} := \mathsf{Compress}_q(\mathbf{A}\mathbf{s} + \mathbf{e}, d_t)$
5: **return** $(pk := (\mathbf{t}, \rho), sk := \mathbf{s})$

Algorithm 2 Kyber.CPA.Enc($pk = (\mathbf{t}, \rho), m \in \mathcal{M}$): encryption

$r \leftarrow \{0,1\}^{256}$
$\mathbf{t} := \mathsf{Decompress}_q(\mathbf{t}, d_t)$
$\mathbf{A} \sim R_q^{k \times k} := \mathsf{Sam}(\rho)$
$(\mathbf{r}, \mathbf{e}_1, e_2) \sim \beta_\eta^k \times \beta_\eta^k \times \beta_\eta := \mathsf{Sam}(r)$
$\mathbf{u} := \mathsf{Compress}_q(\mathbf{A}^T\mathbf{r} + \mathbf{e}_1, d_u)$
$v := \mathsf{Compress}_q\left(\mathbf{t}^T\mathbf{r} + e_2 + \left\lceil \frac{q}{2} \right\rfloor \cdot m, d_v\right)$
return $c := (\mathbf{u}, v)$

Algorithm 3 Kyber.CPA.Dec($sk = \mathbf{s}, c = (\mathbf{u}, v)$): decryption

$\mathbf{u} := \mathsf{Decompress}_q(\mathbf{u}, d_u)$
$v := \mathsf{Decompress}_q(v, d_v)$
return $\mathsf{Compress}_q(v - \mathbf{s}^T\mathbf{u}, 1)$

Kyber.CPA Correctness

	n	k	q	η	(d_u, d_v, d_t)	δ	pq-sec
Kyber	256	3	7681	4	$(11, 3, 11)$	2^{-142}	161

- **Correctness**
  - **We show that Kyber.CPA is $(1-\delta)$-correct with $\delta < 2^{-128}$.**

Theorem 1. *Let k be a positive integer parameter. Let $\mathbf{s}, \mathbf{e}, \mathbf{r}, \mathbf{e}_1, e_2$ be random variables that have the same distribution as in Algorithms 1 and 2. Also, let $\mathbf{c}_t \leftarrow \psi_{d_t}^k, \mathbf{c}_u \leftarrow \psi_{d_u}^k, c_v \leftarrow \psi_{d_v}$ be distributed according to the distribution ψ defined as follows:*
Let ψ_d^k be the following distribution over R:

1: *Choose uniformly-random* $\mathbf{y} \leftarrow R^k$
2: ***return*** $\left(\mathbf{y} - \mathsf{Decompress}_q\left(\mathsf{Compress}_q(\mathbf{y}, d), d\right)\right)$ *mod*$^{\pm}$ q.

Denote

$$\delta = \Pr\left[\left\|\mathbf{e}^T\mathbf{r} + e_2 + \mathbf{c}_v - \mathbf{s}^T\mathbf{e}_1 + \mathbf{c}_t^T\mathbf{r} - \mathbf{s}^T\mathbf{c}_u\right\|_\infty \geq \lceil q/4 \rfloor\right].$$

Then Kyber.CPA *is $(1-\delta)$-correct.*

Kyber.CPA Correctness

	n	k	q	η	(d_u, d_v, d_t)	δ	pq-sec
Kyber	256	3	7681	4	$(11, 3, 11)$	2^{-142}	161

- **Correctness**
  - **We show that Kyber.CPA is $(1-\delta)$-correct with $\delta < 2^{-128}$.**

Remark 1. *We provide with our software a Python script that allows to compute a tight upper bound on δ; the parameter set we recommend for* Kyber *in Table 1 yields* $\delta = 2^{-142}$.

Proof. The value of $\mathbf{t}$ in line 6 of Algorithm 2 is:

$$\mathbf{t} = \mathsf{Decompress}_q\left(\mathsf{Compress}_q(\mathbf{As}+\mathbf{e}, d_t), d_t\right) = \mathbf{As}+\mathbf{e}+\mathbf{c}_t,$$

for some $\mathbf{c}_t \in R^k$. The value of $\mathbf{u}$ in Algorithm 3 is

$$\begin{aligned}\mathbf{u} &= \mathsf{Decompress}_q\left(\mathsf{Compress}_q(\mathbf{A}^T\mathbf{r}+\mathbf{e}_1, d_u), d_u\right)\\ &= \mathbf{A}^T\mathbf{r}+\mathbf{e}_1+\mathbf{c}_u,\end{aligned}$$

for some $\mathbf{c}_u \in R^k$. And the value of v is

Kyber.CPA Correctness

	n	k	q	η	(d_u, d_v, d_t)	δ	pq-sec
Kyber	256	3	7681	4	$(11, 3, 11)$	2^{-142}	161

- **Correctness**
  - **We show that Kyber.CPA is $(1-\delta)$-correct with $\delta < 2^{-128}$.**

Remark 1. *We provide with our software a Python script that allows to compute a tight upper bound on δ; the parameter set we recommend for* Kyber *in Table 1 yields* $\delta = 2^{-142}$.

$$
\begin{aligned}
v &= \mathsf{Decompress}_q\left(\mathsf{Compress}_q(\mathbf{t}^T\mathbf{r} + e_2 + \lceil q/2 \rfloor \cdot m, d_v), d_v\right) \\
&= \mathbf{t}^T\mathbf{r} + e_2 + \lceil q/2 \rfloor \cdot m + \mathbf{c}_v \\
&= (\mathbf{A}\mathbf{s} + \mathbf{e} + \mathbf{c}_t)^T\mathbf{r} + e_2 + \lceil q/2 \rfloor \cdot m + c_v \\
&= (\mathbf{A}\mathbf{s} + \mathbf{e})^T\mathbf{r} + e_2 + \lceil q/2 \rfloor \cdot m + c_v + \mathbf{c}_t^T\mathbf{r},
\end{aligned}
$$

for some $c_v \in R$. In all of the above, we can safely

Kyber.CPA Correctness

	n	k	q	η	(d_u, d_v, d_t)	δ	pq-sec
Kyber	256	3	7681	4	$(11, 3, 11)$	2^{-142}	161

- **Correctness**
  - **We show that Kyber.CPA is $(1-\delta)$-correct with $\delta < 2^{-128}$.**

Remark 1. *We provide with our software a Python script that allows to compute a tight upper bound on δ; the parameter set we recommend for* Kyber *in Table 1 yields* $\delta = 2^{-142}$.

for some $c_v \in R$. In all of the above, we can safely assume that the values $\mathbf{c}_t, \mathbf{c}_u$, and c_v are distributed according to the distribution ψ defined in the theorem statement. The reason is that all of these are of the form $\big(\mathbf{y} - \mathsf{Decompress}_q\big(\mathsf{Compress}_q(\mathbf{y}, d), d\big)\big) \bmod^{\pm} q$ where $\mathbf{y}$ is pseudo-random based on the hardness of Module-LWE.

Using the above, we obtain

$$v - \mathbf{s}^T\mathbf{u} = \mathbf{e}^T\mathbf{r} + e_2 + \lceil q/2 \rfloor \cdot m + \mathbf{c}_v + \mathbf{c}_t^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1 - \mathbf{s}^T\mathbf{c}_u$$

Kyber.CPA Correctness

	n	k	q	η	(d_u, d_v, d_t)	δ	pq-sec
Kyber	256	3	7681	4	$(11, 3, 11)$	2^{-142}	161

- **Correctness**
  - **We show that Kyber.CPA is $(1-\delta)$-correct with $\delta < 2^{-128}$.**

Remark 1. *We provide with our software a Python script that allows to compute a tight upper bound on δ; the parameter set we recommend for* Kyber *in Table 1 yields* $\delta = 2^{-142}$.

If $\left\|\mathbf{e}^T\mathbf{r} + e_2 + \mathbf{c}_v + \mathbf{c}_t^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1 - \mathbf{s}^T\mathbf{c}_u\right\|_\infty < \lceil q/4 \rfloor$, then we can write $v - \mathbf{s}^T\mathbf{u} = w + \lceil q/2 \rfloor \cdot m$ where $\|w\|_\infty < \lceil q/4 \rfloor$. Define $m' = \mathsf{Compress}_q(v - \mathbf{s}^T\mathbf{u}, 1)$. We then know that

$$\begin{aligned} \lceil q/4 \rfloor &\geq \|v - \mathbf{s}^T\mathbf{u} - \lceil q/2 \rfloor \cdot m'\|_\infty \\ &= \|w + \lceil q/2 \rfloor \cdot m - \lceil q/2 \rfloor \cdot m'\|_\infty. \end{aligned}$$

Kyber.CPA Correctness

	n	k	q	η	(d_u, d_v, d_t)	δ	pq-sec
Kyber	256	3	7681	4	$(11, 3, 11)$	2^{-142}	161

- **Correctness**
  - **We show that Kyber.CPA is $(1-\delta)$-correct with $\delta < 2^{-128}$.**

Remark 1. *We provide with our software a Python script that allows to compute a tight upper bound on δ; the parameter set we recommend for* Kyber *in Table 1 yields* $\delta = 2^{-142}$.

By the triangle inequality and the fact that $\|w\|_\infty < \lceil q/4 \rfloor$, we obtain

$$\|\lceil q/2 \rfloor \cdot (m - m')\|_\infty < 2 \cdot \lceil q/4 \rfloor,$$

which (for all odd q) implies that $m = m'$, and proves the correctness of Kyber.CPA. □

Kyber.CPA Security

- **Security of a modified scheme**
  - **Kyber.CPA′ : without compressing t (Algorithm 1 Line 4)**

    (Algorithm 2 Line 2)
  - **Kyber.CPA′ is IND-CPA secure under the Module-LWE hardness assumption.**

Theorem 2. *For any adversary* A*, there exists an adversary* B *such that* $\mathbf{Adv}^{\mathrm{cpa}}_{\mathsf{Kyber.CPA}'}(\mathsf{A}) \leq 2 \cdot \mathbf{Adv}^{\mathrm{mlwe}}_{k+1,k,\eta}(\mathsf{B})$.

Kyber.CPA Security

Theorem 2. *For any adversary* A*, there exists an adversary* B *such that* $\mathbf{Adv}^{\mathrm{cpa}}_{\mathsf{Kyber.CPA}'}(\mathsf{A}) \leq 2 \cdot \mathbf{Adv}^{\mathrm{mlwe}}_{k+1,k,\eta}(\mathsf{B})$.

$$\mathbf{Adv}^{\mathrm{mlwe}}_{m,k,\eta}(\mathsf{A}) = \left| \Pr\left[b' = 1 \ : \ \begin{array}{l} \mathbf{A} \leftarrow R_q^{m \times k}; (\mathbf{s}, \mathbf{e}) \leftarrow \beta_\eta^k \times \beta_\eta^m; \\ \mathbf{b} = \mathbf{As} + \mathbf{e}; b' \leftarrow \mathsf{A}(\mathbf{A}, \mathbf{b}) \end{array} \right] - \Pr\left[b' = 1 \ : \ \mathbf{A} \leftarrow R_q^{m \times k}; \mathbf{b} \leftarrow R_q^m; b' \leftarrow \mathsf{A}(\mathbf{A}, \mathbf{b}) \right] \right|.$$

- **Security of a modified scheme**

Proof. Let A be an adversary that is CPA security experiment which we call game G_0, i.e., $\mathbf{Adv}^{\mathrm{cpa}}_{\mathsf{Kyber.CPA}'}(\mathsf{A}) = |\Pr[b = b' \text{ in game } G_0] - 1/2|$. In game G_1, the value $\mathbf{t}' := \mathbf{As} + \mathbf{e}$ which is used in KeyGen is substituted by a uniform random value. It is possible to verify that there exists an adversary B with the same running time as that of A such that $|\Pr[b = b' \text{ in game } G_0] - |\Pr[b = b' \text{ in game } G_1]| \leq \mathbf{Adv}^{\mathrm{mlwe}}_{k,k,\eta}(\mathsf{B}) \leq \mathbf{Adv}^{\mathrm{mlwe}}_{k+1,k,\eta}(\mathsf{B})$. In game G_2, the values $\mathbf{u}' := \mathbf{A}^T\mathbf{r} + \mathbf{e}_1$ and $v' := \mathbf{t}^T\mathbf{r} + e_2$ used in the generation of the challenge ciphertext are simultaneously substituted with uniform random values. Again, there exists an adversary B with the same running time as that of A with $|\Pr[b = b' \text{ in game } G_1] - |\Pr[b = b' \text{ in game } G_2]| \leq \mathbf{Adv}^{\mathrm{mlwe}}_{k+1,k,\eta}(\mathsf{B})$. Note that in game G_2, the value v from the challenge ciphertext is independent of bit b and therefore $\Pr[b = b' \text{ in game } G_2] = 1/2$. Collecting the probabilities yields the required bound. □

Kyber.CPA Security

- **Security of the real scheme**

Security of the real scheme. In the real scheme, $\mathbf{t} := \mathsf{Decompress}_q\big(\mathsf{Compress}_q(\mathbf{t}, d_u), d_u\big)$ in Line 2 of Algorithm 2 is no longer uniform in R_q^k, and so one cannot conclude that the distribution $(\mathbf{A}^T\mathbf{r} + \mathbf{e}_1, \mathbf{t}^T\mathbf{r} + e_2)$, for $\mathbf{r}, \mathbf{e}_1 \sim \beta_\eta^k$ and $e_2 \sim \beta_\eta$ is indistinguishable from uniform based on the hardness of the Module-LWE problem. One

Kyber.CPA Security

- **Security of the real scheme**

based on the hardness of the Module-LWE problem. One way to fix this issue is for the encryptor to add a small random (possibly even public) noise $\mathbf{e}' \in R^k$ to $\mathbf{t}$ such that $\mathbf{t} + \mathbf{e}'$ is uniformly random in R_q^k. The distribution of this noise would have to depend on $\mathbf{t}$ and d_u. For our value of $d_u = 11$, this would involve selecting the coefficients of $\mathbf{e}'$ from the set $\{-1, 0, 1, 2\}$ with a probability distribution that depends on $\mathbf{t}$.[6] To achieve the same distribution more easily, one could instead define the $\mathsf{Compress}_q$ function as truncating the last 2 bits, and the $\mathsf{Decompress}_q$ function as a multiplication by 4. Then the coefficients of $\mathbf{e}'$ can be chosen uniformly at random from $\{0, 1, 2, 3\}$. The main disadvantage of adding the extra $\mathbf{e}'$ is that the term $\mathbf{e}'^T\mathbf{r}$ will appear in the decryption and will add to the decryption error. For the recommended parameters (see Table 1), the decryption error will increase from 2^{-142} to 2^{-121}.

Kyber.CPA Security

- **Security of the real scheme**

Due to the increase in the decryption error and the cumbersome nature of adding the extra error term, we choose to define Kyber to include the compression of the public key $\mathbf{t}$ but not add any noise. There are several reasons why we strongly believe that this choice does not affect security. First, note that in the encryption Algorithm 2, the decompressed value of $\mathbf{t}$ is used in Line 6 and is compressed with $d_v = 3$ (see Table 1 and Table 2). This means that only approximately the highest 3 bits of $\mathbf{t}^T\mathbf{r} + e_2 + \lceil \frac{q}{2} \rfloor \cdot m$ are output per coefficient. In particular, if the modulus q is large enough, then the two distributions are statistically close because

$$\begin{aligned}&\mathsf{Compress}_q\left((\mathbf{t}+\mathbf{e}')^T\mathbf{r} + e_2 + \left\lceil \frac{q}{2} \right\rfloor \cdot m, d_v\right)\\ &\qquad = \mathsf{Compress}_q\left(\mathbf{t}^T\mathbf{r} + e_2 + \left\lceil \frac{q}{2} \right\rfloor \cdot m, d_v\right),\end{aligned}$$

where $\mathbf{e}'$ is chosen so as to make $\mathbf{t}+\mathbf{e}'$ uniform as discussed above. This is exactly the same relationship that exists

Kyber.CPA Security

- **Security of the real scheme**

above. This is exactly the same relationship that exists between the Module-LWE and Module-LWR problems.[7] For certain parameters, the two problems are equivalent (see [18] for the most "liberal" reduction between the two), yet for ones used in practice (c.f. [10] and many submissions to the NIST post-quantum call), it is still assumed that the distribution of Module-LWR is pseudorandom despite the fact that the proof in [18] is no longer applicable.

Additionally, our proposal for the KEM is the CCA-transformation in Algorithm 4 which does not even require that the output of the CPA-secure scheme be pseudo-random since the shared key is formed by using the message m inside a random oracle. Because the entropy of $\mathbf{r}$ is larger than the entropy of v (by a factor larger than 2.5), it implies that it is not enough to only look at the v term to recover m (or perhaps to even distinguish it from uniform), but one must also somehow use the "properly formed" Module-LWE component $\mathbf{u}$ at the same time. We believe that it is extremely unlikely that anything about the message m can be recovered from this information assuming that Module-LWE is hard.

The CCA-secure KEM

Fujisaki-Okamoto(FO) transform

- **CPA-secure PKE → CCA-secure KEM** $PKE\ \mathsf{P} = (\text{Keygen}, \text{Encr}, \text{Decr})$

F : Key derivation function
prf : Pseudorandom function

Keygen():

1 $(\text{pk}, \text{sk}_\mathsf{P}) \leftarrow \text{Keygen}_\mathsf{P}()$
2 $\text{prfk} \xleftarrow{\$} \mathcal{K}_\mathsf{F}$
3 $\text{sk} \leftarrow (\text{sk}_\mathsf{P}, \text{prfk})$
4 return (pk, sk)

Encaps(pk):

1 $m \xleftarrow{\$} \mathcal{M}$
2 $r \leftarrow G(m)$
3 $c \leftarrow \text{Encr}(\text{pk}, m; r)$
4 $K \leftarrow H(m, c)$
5 return (K, c)

Decaps(sk, c):

1 parse $\text{sk} = (\text{sk}_\mathsf{P}, \text{prfk})$
2 $m' \leftarrow \text{Decr}(\text{sk}_\mathsf{P}, c)$
3 if $m' = \perp$:
4 return $\mathsf{F}(\text{prfk}, c)$
5 else if $\text{Encr}(\text{pk}, m'; G(m')) \neq c$:
6 return $\mathsf{F}(\text{prfk}, c)$
7 else: return $H(m', c)$

Fig. 1. Transform $\mathsf{FO}^{\not\perp}(\mathsf{P}, \mathsf{F}, G, H) := (\text{Keygen}, \text{Encaps}, \text{Decaps})$.

Keygen():

1 $(\text{pk}, \text{sk}_\mathsf{P}) \leftarrow \text{Keygen}_\mathsf{P}()$
2 $\text{prfk} \xleftarrow{\$} \mathcal{K}_\mathsf{F}$
3 $\text{sk} \leftarrow (\text{sk}_\mathsf{P}, \text{pk}, H_3(\text{pk}), \text{prfk})$
4 return (pk, sk)

Encaps(pk):

1 $m \xleftarrow{\$} \mathcal{M}$
2 $(\tilde{K}, r) \leftarrow H_1(m, H_3(\text{pk}))$
3 $c \leftarrow \text{Encr}(\text{pk}, m; r)$
4 $K \leftarrow H(\tilde{K}, H_2(c))$
5 return (K, c)

Decaps(sk, c):

1 parse $\text{sk} = (\text{sk}_\mathsf{P}, \text{pk}, h_{\text{pk}}, \text{prfk})$
2 $m' \leftarrow \text{Decr}(\text{sk}_\mathsf{P}, c)$
3 if $m' = \perp$:
4 return $\mathsf{F}(\text{prfk}, H_2(c))$
5 else if $\text{Encr}(\text{pk}, m'; (H_1(m', h_{\text{pk}}))_2) \neq c$:
6 return $\mathsf{F}(\text{prfk}, H_2(c))$
7 else: return $H((H_1(m'))_1, H_2(c))$

Fig. 2. Transform $\mathsf{FO}^{\text{Kyber}}(\mathsf{P}, \mathsf{F}, H, H_1, H_2) := (\text{Keygen}, \text{Encaps}, \text{Decaps})$. Modifications in blue.

[1] Secure integration of asymmetric and symmetric encryption schemes. 1999
[2] A modular analysis of the Fujisaki-Okamoto transformation. 2017

Kyber's FO transform

- **CPA-secure PKE → CCA-secure KEM**

Keygen():

1 $(\mathrm{pk}, \mathrm{sk}_{\mathsf{P}}) \leftarrow \mathrm{Keygen}_{\mathsf{P}}()$
2 $\mathrm{prfk} \xleftarrow{\$} \mathcal{K}_{\mathsf{F}}$
3 $\mathrm{sk} \leftarrow (\mathrm{sk}_{\mathsf{P}}, \mathrm{pk}, H_3(\mathrm{pk}), \mathrm{prfk})$
4 return $(\mathrm{pk}, \mathrm{sk})$

Encaps(pk):

1 $m \xleftarrow{\$} \mathcal{M}$
2 $(\tilde{K}, r) \leftarrow H_1(m, H_3(\mathrm{pk}))$
3 $c \leftarrow \mathrm{Encr}(\mathrm{pk}, m; r)$
4 $K \leftarrow H(\tilde{K}, H_2(c))$
5 return (K, c)

Decaps(sk, c):

1 parse $\mathrm{sk} = (\mathrm{sk}_{\mathsf{P}}, \mathrm{pk}, h_{\mathrm{pk}}, \mathrm{prfk})$
2 $m' \leftarrow \mathrm{Decr}(\mathrm{sk}_{\mathsf{P}}, c)$
3 if $m' = \perp$:
4 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
5 else if $\mathrm{Encr}(\mathrm{pk}, m'; (H_1(m', h_{\mathrm{pk}}))_2) \neq c$:
6 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
7 else: return $H((H_1(m'))_1, H_2(c))$

Fig. 2. Transform $\mathsf{FO}^{\mathsf{Kyber}}(\mathsf{P}, \mathsf{F}, H, H_1, H_2) := (\mathrm{Keygen}, \mathrm{Encaps}, \mathrm{Decaps})$. Modifications in blue.

Algorithm 1 Kyber.CPA.KeyGen(): key generation

1: $\rho, \sigma \leftarrow \{0,1\}^{256}$
2: $\mathbf{A} \sim R_q^{k \times k} := \mathsf{Sam}(\rho)$
3: $(\mathbf{s}, \mathbf{e}) \sim \beta_\eta^k \times \beta_\eta^k := \mathsf{Sam}(\sigma)$
4: $\mathbf{t} := \mathsf{Compress}_q(\mathbf{As} + \mathbf{e}, d_t)$
5: **return** $(pk := (\mathbf{t}, \rho), sk := \mathbf{s})$

Kyber's FO transform

- **CPA-secure PKE → CCA-secure KEM**

Keygen():

1 $(\mathrm{pk}, \mathrm{sk}_{\mathsf{P}}) \leftarrow \mathrm{Keygen}_{\mathsf{P}}()$
2 $\mathrm{prfk} \xleftarrow{\$} \mathcal{K}_{\mathsf{F}}$
3 $\mathrm{sk} \leftarrow (\mathrm{sk}_{\mathsf{P}}, \mathrm{pk}, H_3(\mathrm{pk}), \mathrm{prfk})$
4 return $(\mathrm{pk}, \mathrm{sk})$

Encaps(pk):

1 $m \xleftarrow{\$} \mathcal{M}$
2 $(\tilde{K}, r) \leftarrow H_1(m, H_3(\mathrm{pk}))$
3 $c \leftarrow \mathrm{Encr}(\mathrm{pk}, m; r)$
4 $K \leftarrow H(\tilde{K}, H_2(c))$
5 return (K, c)

Decaps(sk, c):

1 parse $\mathrm{sk} = (\mathrm{sk}_{\mathsf{P}}, \mathrm{pk}, h_{\mathrm{pk}}, \mathrm{prfk})$
2 $m' \leftarrow \mathrm{Decr}(\mathrm{sk}_{\mathsf{P}}, c)$
3 if $m' = \perp$:
4 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
5 else if $\mathrm{Encr}(\mathrm{pk}, m'; (H_1(m', h_{\mathrm{pk}}))_2) \neq c$:
6 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
7 else: return $H((H_1(m'))_1, H_2(c))$

Fig. 2. Transform $\mathsf{FO}^{\mathsf{Kyber}}(\mathsf{P}, \mathsf{F}, H, H_1, H_2) := (\mathrm{Keygen}, \mathrm{Encaps}, \mathrm{Decaps})$. Modifications in blue.

Algorithm 4 Kyber.Encaps$(pk = (\mathbf{t}, \rho))$

1: $m \leftarrow \{0,1\}^{256}$
2: $(\hat{K}, r) := \mathsf{G}(\mathsf{H}(pk), m)$
3: $(\mathbf{u}, v) := \mathsf{Kyber.CPA.Enc}\,((\mathbf{t}, \rho), m; r)$
4: $c := (\mathbf{u}, v)$
5: $K := \mathsf{H}(\hat{K}, \mathsf{H}(c))$
6: **return** (c, K)

Kyber's FO transform

- **CPA-secure PKE → CCA-secure KEM**

Keygen():

1 $(\mathrm{pk}, \mathrm{sk}_{\mathsf{P}}) \leftarrow \mathrm{Keygen}_{\mathsf{P}}()$
2 $\mathrm{prfk} \xleftarrow{\$} \mathcal{K}_{\mathsf{F}}$
3 $\mathrm{sk} \leftarrow (\mathrm{sk}_{\mathsf{P}}, \mathrm{pk}, H_3(\mathrm{pk}), \mathrm{prfk})$
4 return $(\mathrm{pk}, \mathrm{sk})$

Encaps(pk):

1 $m \xleftarrow{\$} \mathcal{M}$
2 $(\tilde{K}, r) \leftarrow H_1(m, H_3(\mathrm{pk}))$
3 $c \leftarrow \mathrm{Encr}(\mathrm{pk}, m; r)$
4 $K \leftarrow H(\tilde{K}, H_2(c))$
5 return (K, c)

Decaps(sk, c):

1 parse $\mathrm{sk} = (\mathrm{sk}_{\mathsf{P}}, \mathrm{pk}, h_{\mathrm{pk}}, \mathrm{prfk})$
2 $m' \leftarrow \mathrm{Decr}(\mathrm{sk}_{\mathsf{P}}, c)$
3 if $m' = \perp$:
4 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
5 else if $\mathrm{Encr}(\mathrm{pk}, m'; (H_1(m', h_{\mathrm{pk}}))_2) \neq c$:
6 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
7 else: return $H((H_1(m'))_1, H_2(c))$

Fig. 2. Transform $\mathsf{FO}^{\mathsf{Kyber}}(\mathsf{P}, \mathsf{F}, H, H_1, H_2) := (\mathrm{Keygen}, \mathrm{Encaps}, \mathrm{Decaps})$. Modifications in blue.

Algorithm 5 Kyber.Decaps$(sk = (\mathbf{s}, z, \mathbf{t}, \rho), c = (\mathbf{u}, v))$

1: $m' := \mathsf{Kyber.CPA.Dec}(\mathbf{s}, (\mathbf{u}, v))$
2: $(\hat{K}', r') := \mathrm{G}(\mathrm{H}(pk), m')$
3: $(\mathbf{u}', v') := \mathsf{Kyber.CPA.Enc}\,((\mathbf{t}, \rho), m'; r')$
4: **if** $(\mathbf{u}', v') = (\mathbf{u}, v)$ **then**
5: **return** $K := \mathrm{H}(\hat{K}', \mathrm{H}(c))$
6: **else**
7: **return** $K := \mathrm{H}(z, \mathrm{H}(c))$
8: **end if**

CCA-secure KEM

Let $G\colon \{0,1\}^* \to \{0,1\}^{2\times 256}$ and $H\colon \{0,1\}^* \to \{0,1\}^{256}$ be hash functions.

- **Correctness**

Correctness. If Kyber.CPA is $(1-\delta)$-correct and G is a random oracle, then Kyber is $(1-\delta)$-correct [44].

- **PKE correct**
  - **public-key encryption scheme PKE is $(1-\delta)$-correct if**

$$\mathbf{E}[\max_{m\in\mathcal{M}} \Pr[\mathsf{Dec}(sk, \mathsf{Enc}(pk, m)) = m]] \geq 1-\delta,$$

We say that KEM is $(1-\delta)$-correct if $\Pr[\mathsf{Decaps}(sk, c) = K : (c, K) \leftarrow \mathsf{Encaps}(pk)] \geq 1-\delta$, where the probability is taken over $(pk, sk) \leftarrow \mathsf{KeyGen}()$ and the random coins of Encaps.

We recall the standard security notion for key encapsulation of indistinguishability under chosen-ciphertext attack. The advantage of an adversary A is defined as $\mathbf{Adv}^{\mathrm{cca}}_{\mathsf{KEM}}(\mathsf{A}) =$

$$\left| \Pr\left[b = b' \;:\; \begin{array}{l} (pk, sk) \leftarrow \mathsf{KeyGen}(); \\ b \leftarrow \{0,1\}; \\ (c^*, K_0^*) \leftarrow \mathsf{Encaps}(pk); \\ K_1^* \leftarrow \mathcal{K}; \\ b' \leftarrow \mathsf{A}^{\mathrm{DECAPS}(\cdot)}(pk, c^*, K_b^*) \end{array} \right] - \frac{1}{2} \right| ,$$

CCA-secure KEM

- **Security**

Security. The following concrete security statement proves Kyber's CCA-security when the hash functions G and H are modeled as random oracles. We provide the concrete security bounds from [44] which considers the KEM variant of the FO transformation and also takes a non-zero correctness error δ into account.

Theorem 3. *For any classical adversary* A *that makes at most* q_{RO} *many queries to random oracles H and G, and* q_D *queries to the decryption oracle, there exists an adversary* B *such that*

$$\mathbf{Adv}^{\mathrm{cca}}_{\mathsf{Kyber}}(\mathsf{A}) \leq 3\mathbf{Adv}^{\mathrm{cpa}}_{\mathsf{Kyber.CPA}}(\mathsf{B}) + q_{RO} \cdot \delta + \frac{3q_{RO}}{2^{256}}.$$

Theorem 4. *For any quantum adversary* A *that makes at most* q_{RO} *many queries to quantum random oracles H and G, and at most* q_D *many (classical) queries to the decryption oracle, there exists a quantum adversary* B *such that*

$$\mathbf{Adv}^{\mathrm{cca}}_{\mathsf{Kyber}}(\mathsf{A}) \leq 8q^2_{RO} \cdot \delta + 4q_{RO} \cdot \sqrt{\mathbf{Adv}^{\mathrm{pr}}_{\mathsf{Kyber.CPA}}(\mathsf{B})}.$$

CCA-secure KEM

- **Security**

Keygen():	Encaps(pk):	Decaps(sk, c):
1 $(\mathsf{pk}, \mathsf{sk}_\mathsf{P}) \leftarrow \mathrm{Keygen}_\mathsf{P}()$	1 $m \xleftarrow{\$} \mathcal{M}$	1 parse $\mathsf{sk} = (\mathsf{sk}_\mathsf{P}, \mathsf{prfk})$
2 $\mathsf{prfk} \xleftarrow{\$} \mathcal{K}_\mathsf{F}$	2 $\tilde{K} \leftarrow H_1'(m)$	2 $m' \leftarrow \mathrm{Decr}(\mathsf{sk}_\mathsf{P}, c)$
3 $\mathsf{sk} \leftarrow (\mathsf{sk}_\mathsf{P}, \mathsf{prfk})$	3 $c \leftarrow \mathrm{Encr}(\mathsf{pk}, m)$	3 if $m' = \perp$:
4 return $(\mathsf{pk}, \mathsf{sk})$	4 $K \leftarrow H(\tilde{K}, H_2(c))$	4 return $\mathsf{F}(\mathsf{prfk}, H_2(c))$
	5 return (K, c)	5 else if $\mathrm{Encr}(\mathsf{pk}, m') \neq c$:
		6 return $\mathsf{F}(\mathsf{prfk}, H_2(c))$
		7 else
		8 return $H(H_1'(m'), H_2(c))$

Fig. 4. Transform $\mathsf{U}^{\mathsf{Kyber}}(\mathsf{P}, \mathsf{F}, H, H_1', H_2) := (\mathrm{Keygen}, \mathrm{Encaps}, \mathrm{Decaps})$.

Game 0 (IND-CCA). *This is the original* IND-CCA *game against* $\mathsf{U}^{\mathsf{Kyber}}(\mathsf{P}, \mathsf{F}, H, H_1', H_2)$.

By definition we have $\mathrm{Adv}^{\text{IND-CCA}}_{\mathsf{U}^{\mathsf{Kyber}}(\mathsf{P})}(\mathcal{A}) = \left|W_0 - \frac{1}{2}\right|$.

Game 1 (PRF is random). *Game 1 is the same as Game 0, except we replace* $\mathsf{F}(\mathsf{prfk}, \cdot)$ *with a random function* $R \xleftarrow{\$} \mathcal{K}^{\mathcal{C}_H}$.

We construct a PRF-adversary $\mathcal{B}_1$ which simulates Game 1 replacing calls to $\mathsf{F}(\mathsf{prfk}, \cdot)$ by calls to its oracle, runs $\mathcal{A}$, and outputs 1 if $\mathcal{A}$ wins and 0 otherwise. Now, by construction

$$\Pr_{k \xleftarrow{\$} \mathcal{K}}\left[\mathcal{B}_1^{\mathsf{F}(k,\cdot)} = 1\right] = W_0 \qquad \Pr_{R \xleftarrow{\$} \mathcal{K}^{\mathcal{C}_{H_2}}}\left[\mathcal{B}_1^{R(\cdot)} = 1\right] = W_1\,.$$

Hence,

$$|W_1 - W_0| = \mathrm{Adv}^{\mathrm{PRF}}_{\mathsf{F}}(\mathcal{B}_1).$$

CCA-secure KEM

- **Security**

Keygen():

1 $(\mathrm{pk}, \mathrm{sk}_{\mathsf{P}}) \leftarrow \mathrm{Keygen}_{\mathsf{P}}()$
2 $\mathrm{prfk} \xleftarrow{\$} \mathcal{K}_{\mathsf{F}}$
3 $\mathrm{sk} \leftarrow (\mathrm{sk}_{\mathsf{P}}, \mathrm{prfk})$
4 return $(\mathrm{pk}, \mathrm{sk})$

Encaps(pk):

1 $m \xleftarrow{\$} \mathcal{M}$
2 $\tilde{K} \leftarrow H_1'(m)$
3 $c \leftarrow \mathrm{Encr}(\mathrm{pk}, m)$
4 $K \leftarrow H(\tilde{K}, H_2(c))$
5 return (K, c)

Decaps(sk, c):

1 parse $\mathrm{sk} = (\mathrm{sk}_{\mathsf{P}}, \mathrm{prfk})$
2 $m' \leftarrow \mathrm{Decr}(\mathrm{sk}_{\mathsf{P}}, c)$
3 if $m' = \perp$:
4 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
5 else if $\mathrm{Encr}(\mathrm{pk}, m') \neq c$:
6 return $\mathsf{F}(\mathrm{prfk}, H_2(c))$
7 else
8 return $H(H_1'(m'), H_2(c))$

Fig. 4. Transform $\mathsf{U}^{\not\perp}[\mathsf{P}, \mathsf{F}, H, H_1', H_2] := (\mathrm{Keygen}, \mathrm{Encaps}, \mathrm{Decaps})$.

Game 2 (Replace H_1' by random permutation π). *Game 2 is the same as Game 1, but the game implements H_1' using a random permutation π for which it also knows the inverse π^{-1}.*

If $\mathcal{A}$ could change its behavior from one game to another, we could use $\mathcal{A}$ to distinguish random functions from random permutations. Indeed, one can construct an adversary $\mathcal{C}_2$ that, given oracle access to a function $\mathsf{F} : \mathcal{M} \to \mathcal{M}$ that is sampled uniformly at random from either all functions or a permutation π sampled uniformly at random from the set of all permutations with the respective domain and co-domain, simply sets H_1' as its own oracle and runs $\mathcal{A}$ according to the rules of Game 2. $\mathcal{C}_2$ returns 1 if A wins. If interacting with a random function, we perfectly simulate Game 1, otherwise, Game 2. By [Zha15], there exists a constant C such that the advantage of any q query quantum adversary in distinguishing a random function from a random permutation over $\mathcal{M}$ is upper-bounded by $Cq^3/|\mathcal{M}|$, hence,

$$|W_2 - W_1| \leq \mathrm{Adv}^{\text{PRF-PRP}}_{H_1'}(\mathcal{C}_2) \leq \frac{C(q_1 + q_{\mathrm{dec}} + 1)^3}{|\mathcal{M}|}.$$

CCA-secure KEM

- **Security**

Keygen():	Encaps(pk):	Decaps(sk, c):
1 $(\mathsf{pk}, \mathsf{sk}_\mathsf{P}) \leftarrow \mathrm{Keygen}_\mathsf{P}()$	1 $m \xleftarrow{\$} \mathcal{M}$	1 parse $\mathsf{sk} = (\mathsf{sk}_\mathsf{P}, \mathsf{prfk})$
2 $\mathsf{prfk} \xleftarrow{\$} \mathcal{K}_\mathsf{F}$	2 $\tilde{K} \leftarrow H_1'(m)$	2 $m' \leftarrow \mathrm{Decr}(\mathsf{sk}_\mathsf{P}, c)$
3 $\mathsf{sk} \leftarrow (\mathsf{sk}_\mathsf{P}, \mathsf{prfk})$	3 $c \leftarrow \mathrm{Encr}(\mathsf{pk}, m)$	3 if $m' = \perp$:
4 return $(\mathsf{pk}, \mathsf{sk})$	4 $K \leftarrow H(\tilde{K}, H_2(c))$	4 return $\mathsf{F}(\mathsf{prfk}, H_2(c))$
	5 return (K, c)	5 else if $\mathrm{Encr}(\mathsf{pk}, m') \neq c$:
		6 return $\mathsf{F}(\mathsf{prfk}, H_2(c))$
		7 else
		8 return $H(H_1'(m'), H_2(c))$

Fig. 4. Transform $\mathsf{U}^{\mathsf{Kyber}}(\mathsf{P}, \mathsf{F}, H, H_1', H_2) := (\mathrm{Keygen}, \mathrm{Encaps}, \mathrm{Decaps})$.

$$|W_9 - W_8| \leq \mathrm{Adv}_\mathsf{P}^{\mathsf{FFC}}(\mathcal{B}_9) + \epsilon.$$

$$\left| w_0 - \frac{1}{2} \right| \leq \sqrt{\mathrm{Adv}_\mathsf{P}^{\mathsf{OW\text{-}CPA}}(\mathcal{B})} + \mathrm{Adv}_\mathsf{P}^{\mathsf{FFC}}(\mathcal{B}_9) + \mathrm{Adv}_\mathsf{P}^{\mathsf{FFC}}(\mathcal{B}_5) + 2\epsilon + \mathrm{Adv}_\mathsf{F}^{\mathsf{PRF}}(\mathcal{B}_1) +$$

$$\frac{C(q_1 + q_{\mathrm{dec}} + 1)^3}{|\mathcal{M}|} + \frac{C(q_2 + q_{\mathrm{dec}} + 1)^3}{|\mathcal{C}_H|}$$

CCA-secure KEM

- **Hashing pk into $\hat{K}$**

Algorithm 4 Kyber.Encaps($pk = (\mathbf{t}, \rho)$)

1: $m \leftarrow \{0,1\}^{256}$
2: $(\hat{K}, r) := \mathrm{G}(\mathrm{H}(pk), m)$
3: $(\mathbf{u}, v) := \mathsf{Kyber.CPA.Enc}\,((\mathbf{t}, \rho), m; r)$
4: $c := (\mathbf{u}, v)$
5: $K := \mathrm{H}(\hat{K}, \mathrm{H}(c))$
6: **return** (c, K)

- **First, it makes the KEM contributory; the shared key K does not depend only on input of one of the two parties.**

- **The second effect is a multitarget protection.**

CCA-secure KEM

- **Supporting non-incremental hash APIs.**

Algorithm 4 Kyber.Encaps$(pk = (\mathbf{t}, \rho))$

1: $m \leftarrow \{0,1\}^{256}$
2: $(\hat{K}, r) := \mathrm{G}(\mathrm{H}(pk), m)$
3: $(\mathbf{u}, v) := \mathsf{Kyber.CPA.Enc}\,((\mathbf{t}, \rho), m; r)$
4: $c := (\mathbf{u}, v)$
5: $K := \mathrm{H}(\hat{K}, \mathrm{H}(c))$
6: **return** (c, K)

- **small speedup for decapsulation.**

CCA-secure KEM

- **CCA-secure public-key encryption.**
  - **CCA-secure KEM Kyber + any CCA-secure symmetric encryption scheme**

Algorithm 6 Kyber.Hybrid.KeyGen()

1: $(pk := (\rho, \mathbf{t}), sk := (\mathbf{s}, \rho, \mathbf{t})) \leftarrow$ Kyber.KeyGen()
2: **return** (pk, sk)

Algorithm 7 Kyber.Hybrid.Enc$(pk = (\rho, \mathbf{t}), m)$

1: $(c, K) \leftarrow$ Kyber.Encaps(pk)
2: $c' := E(K, m)$
3: **return** $c'' := (c, c')$

Algorithm 8 Kyber.Hybrid.Dec$(sk = (\mathbf{s}, z, \rho, \mathbf{t}), c'' = (c, c'))$

1: $K :=$ Kyber.Decaps(sk, c)
2: **return** $m := D(K, c')$

Key Exchange Protocols

Key Exchange Protocols

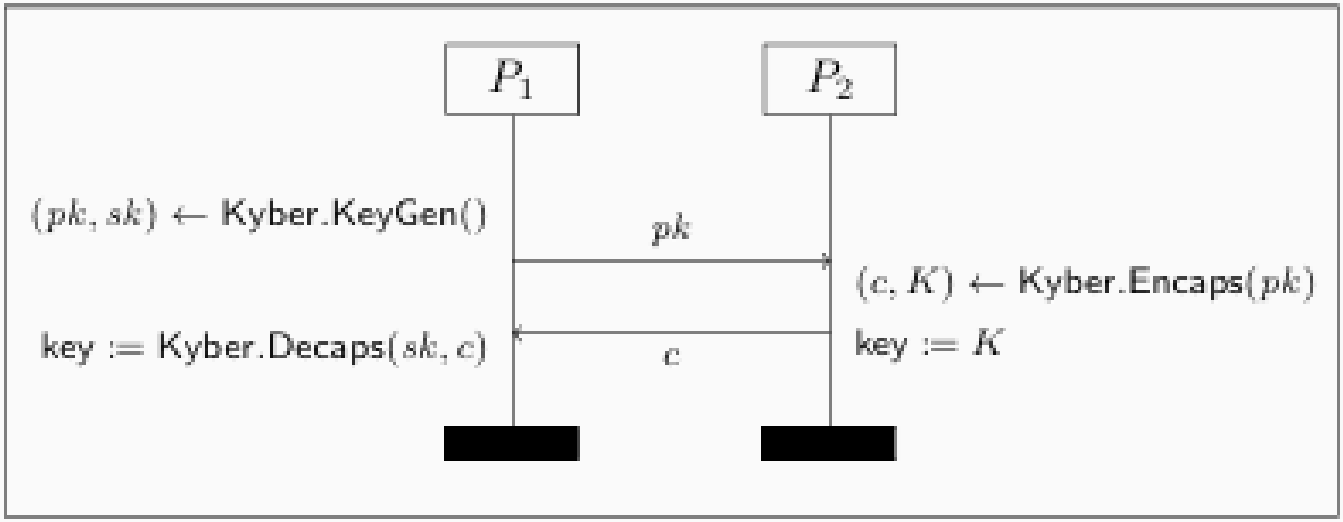


Figure 1. Kyber.KE – Key Exchange protocol using the Kyber $=$ (KeyGen, Encaps, Decaps) key encapsulation mechanism.

Key Exchange Protocols

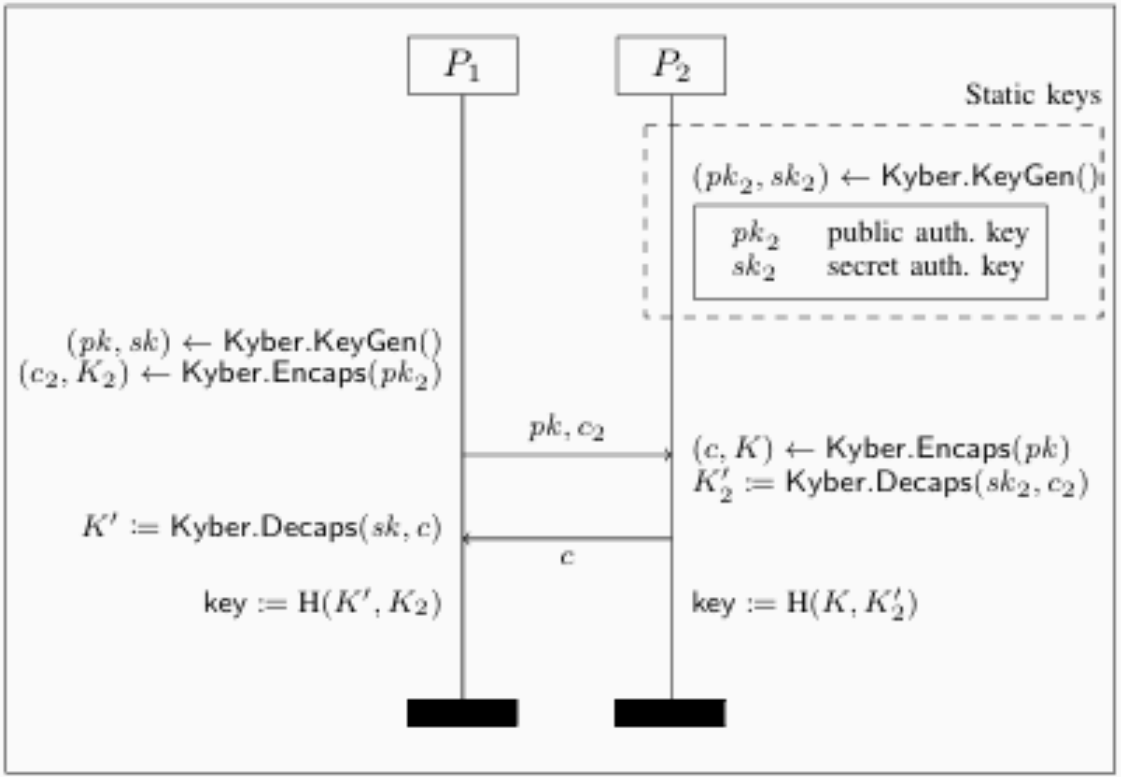


Figure 2. Kyber.UAKE – One-sided authenticated key exchange protocol using Kyber, where P_1 knows the static public key of P_2.

Key Exchange Protocols

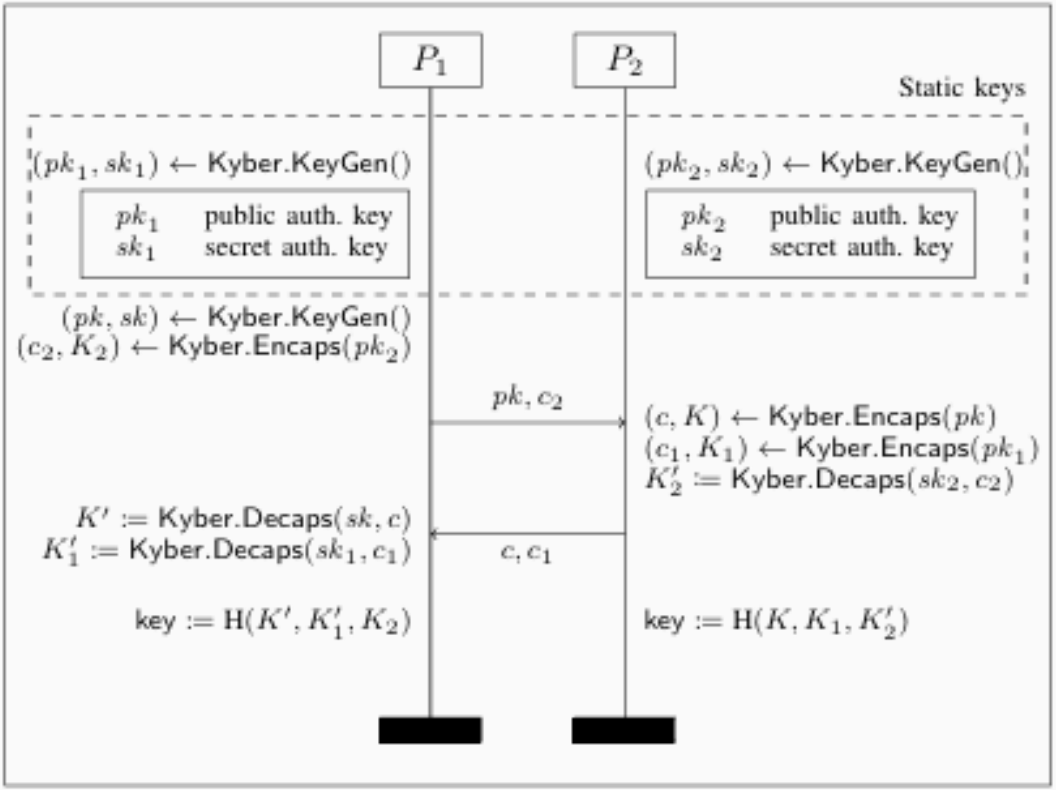


Figure 3. Kyber.AKE – Authenticated key exchange protocol using Kyber, where both parties know each other's static public keys.

감사합니다.